Subscribe to DeepL Pro to translate larger documents.
Visit www.DeepL.com/pro for more information.

RFI
RETE FERROVIARIA ITALIANA
GRUPPO FERROVIE DELLO STATO ITALIANE

Interoperability Specifications for Communication with Field Systems

RFI RETE FERROVIARIA ITALIANA
GRUPPO FERROVIE DELLO STATO ITALIANE

SUMMARY

1. General Concepts 3
1.1. Introduction 3
1.2. Scope, Context, and Objectives 3
1.2.1. Scope 3
1.2.2. Context and Assumptions 4
1.2.3. Objectives of the Specification 6
2. Communication Methods Field System Units– IoT Platform 6
2.1 Authentication of Management Units and Edge 6
2.1.1 Authentication with JWT Tokens 6
2.2 Summary of the Specification 7
2.3 Protocols 8
2.3.1 MQTTS Protocol 8
2.3.2 SFTP Protocol 8
2.3.3 HTTPS Protocol 8
3. Alert Management and Data Flow 9
3.1 Alarm Management 9
3.1.1 Periodic Alarm Recalibration 11
3.2. Fault Management 12
3.2.1 Periodic Fault Realignment 15
3.3 Measurement Management 15
3.3.1 PULL Mode 17
3.4 Snapshot Management 18
3.4.1 Bulk Image 19
3.5 Topics to Use 21
4. Device Management 22
4.1 Master Data Management 22
4.2 Command Management 23
4.3 Topics to use 25
5. Remote access to field devices by RFI 26
6. Remote access to field equipment by Suppliers 26

1. General Concepts

1.1. Introduction

The purpose of this document is to define the management processes and interactions between local OT networks (field-deployed monitoring systems) and the Central IoT Platform, ensuring the completeness and accuracy of the data collected in the field.

The standards covered impact the following operational contexts:

- **Device Management**: management of the **operational configurations of the devices** that make up the physical monitoring infrastructure;
- **Data Flow Management**: processes for **sharing and processing data** recorded and/or processed locally by field systems and transmitted to the Central IoT Platform;
- **Alert Management**: notification of **faults** or **alarms** to the Central IoT Platform in the event of threshold exceedances, malfunctions, or changes in network status.

Please note that, during implementation, the level of coverage for the planned features in each operating context may vary depending on the type of components selected for the specific system; therefore, for example, on less complex networks, a limited set of features may be available.

The objectives to be achieved are as follows:

- Ensure interoperability with the IoT platform, regardless of the type of hardware installed;
- Make measurement data from the field (e.g., alerts) available continuously and promptly (e.g., for real-time alerts);
- Enable the analysis and processing of extremely large volumes of data;
- Enable integrated device management of the ecosystem's systems and devices;
- Ensure the traceability of data sent by OT devices.

1.2. Scope, Context, and Objectives

1.2.1. Scope

RFI's real estate and infrastructure portfolio is diverse and characterized by a variety of construction techniques and materials used over time.

For this reason, it is particularly useful to have a continuous and efficient IoT system that allows for:

- Monitor and verify the condition of infrastructure components;
- Monitor and identify external causes that could lead to potential failures or malfunctions;
- Support the infrastructure inspection and maintenance process.

Generally, an IoT platform consists of three layers:

- EDGE TIER: This layer consists of the OT network, where measurements are collected from the environment, sampled, and transferred to the upper layer according to appropriate logic. This layer includes both the OT network and Edge Gateways for translating, filtering, and sending data to the IoT platform

- PLATFORM TIER: consists of the central IoT platform that performs global operations in collaboration with the Edge Tier, such as device management, stream processing, advanced analytics, and workload orchestration
- ENTERPRISE TIER: consists of the layer where the IoT platform integrates with enterprise IT applications and processes, such as ERP. Enterprise applications, in fact, can benefit from data-driven insights provided by the IoT platform.

(Source: Gartner, "Architecting IoT Using the Gartner Reference Model")

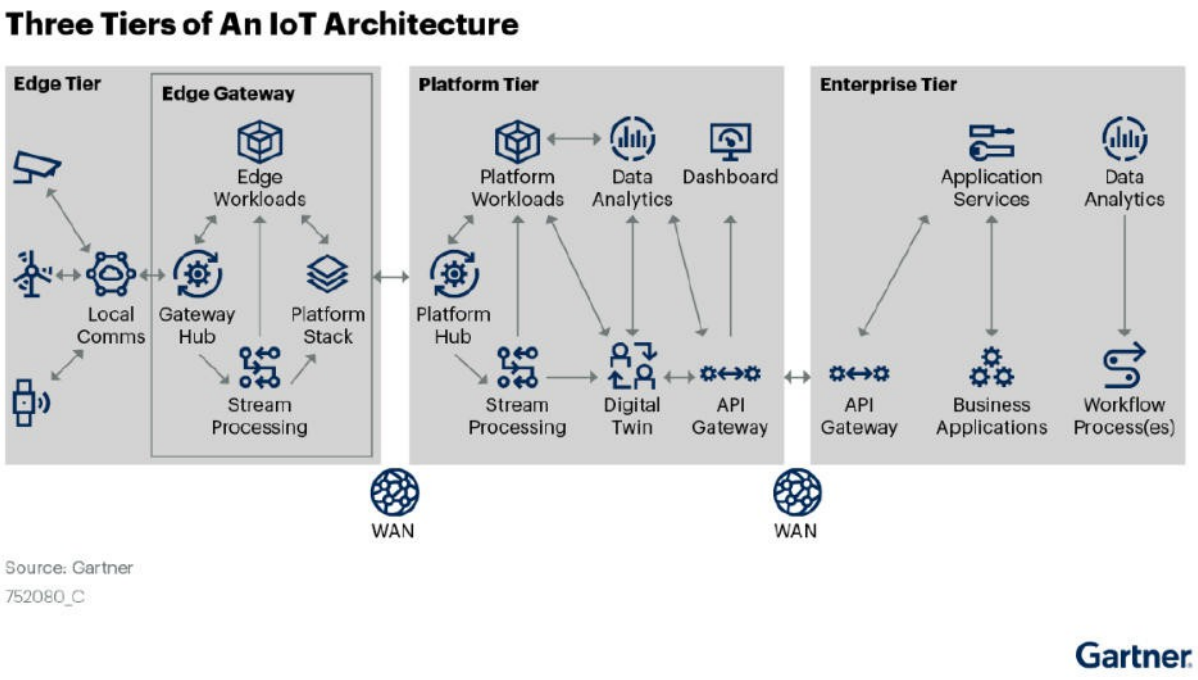

Figure 1 – Three Tiers of an IoT Architecture.

The use of an IoT platform, which operates not only locally but across the entire chain, will enable improved monitoring and maintenance.

For example, by utilizing historical data from field systems, in addition to data recorded by instruments during inspections, it will be possible to integrate the current information assets regarding the condition of the structures to define representative indices, thereby supporting the RFI user in decision-making.

The bridge monitoring model, which will be described below, supported by appropriate technological solutions, will also provide information for real-time monitoring of the structure's condition and for managing incidents and failures.

1.2.2. Context and Assumptions

The context in question involves defining the integration solution between RFI's IoT Platform and the monitoring systems. The logical architecture depicting the bridge monitoring systems is shown below.

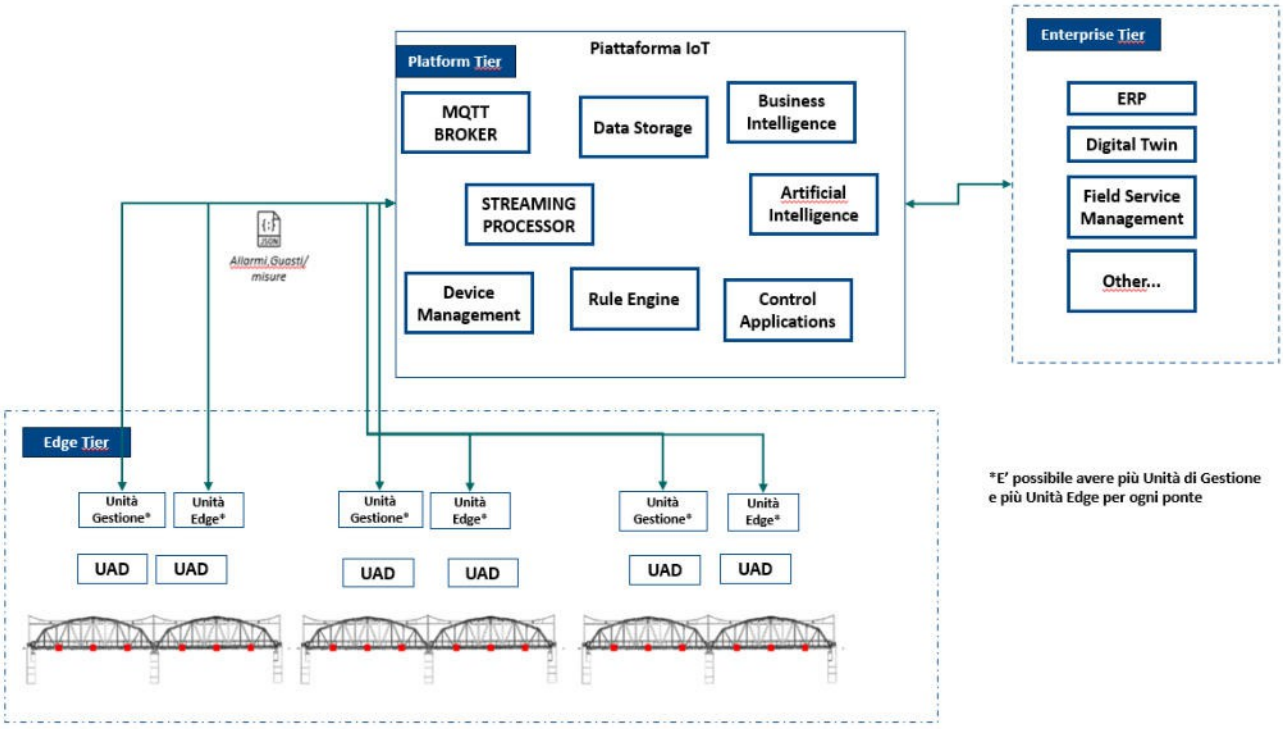

Figure 2 - Logical Architecture

As shown in Figure 2 – Logical Architecture, the data collected by the sensors installed on the bridges is gathered by the Data Acquisition Units (DAUs) to which they are directly connected.

The field architecture includes at least one UAD for each instrumented span, capable of collecting data from the various sensors installed on the specific span being monitored.

Each UAD sends data to a Management Unit. Each Management Unit sends data to one or more Edge Units, consisting of an industrial-grade PC, located inside an outdoor cabinet near the instrumented structure. Depending on the number of spans, each bridge may have multiple Management Units and Edge Units.

The Management Unit manages and configures the UADs connected to it (see Section 5.7 of the Technical Specifications for the Tender), while the Edge Unit is the module in which the algorithms defined by RFI are executed. At the same time, both units will be able to interface with the IoT platform.

Development work on the Edge Unit and the Management Unit will be entirely the responsibility of RFI and its respective suppliers, in accordance with the tender specifications; as of now, FS Technology does not plan to install any products on these components.

The IoT platform provides advanced features including data streaming, logging, and analysis.

The data acquired on the IoT platform will be used for various purposes within the maintenance process:

- Alarm/fault management;
- Optimization of maintenance processes;
- Acquisition and archiving of measurements for detailed analysis of the artwork's condition.

The logical architecture shown (Fig. 2) provides for bidirectional data exchange between the field units (Management and Edge) and the IoT Platform. In fact, there will also be a dedicated *data flow* to the Management Units or Edge Units:

- for transmitting status changes made centrally regarding faults and alarms, respectively;
- to the transmission of commands for managing sensor devices.

1.2.3. Objectives of the Specification

The objective of this specification is to define the communication protocols for integrating the various types and families of data between the Management Units, Edge devices, and RFI's IoT platform. The objectives of the specification are divided into three main functional areas:

- Alert Management - concerns the detection, filtering, and notification of alarm or fault events to the IoT Platform;
- Data Flow Management - refers to the processes and protocols for acquiring and transferring measured values from the field to the centralized data storage, processing, and visualization systems of RFI's IoT platform. This also includes data generated by processing algorithms running at the edge (e.g., structural frequencies, performance indices, etc.)
- Device Management - refers to the management of the status and operational configurations of the devices that make up the physical monitoring infrastructure, such as sensors, data acquisition systems, and field computers (e.g., Management Units, Edge Units, UADs, data loggers, local data storage units, etc.)

The specifications set forth in this document are intended to serve as examples and are provided in considerable detail; however, they are not exhaustive, as they will be further refined during the implementation phase. Therefore, the possibility of changes cannot be ruled out. Any such changes will be communicated prior to the start of development of the field systems and will not alter the nature or *scope* of work expected of the contractors responsible for the field systems.

2. Communication Methods Between Field System Units and the IoT Platform

2.1 Authentication of Management Units and Edge Units

Authentication will involve the use of a TLS (X.509) digital certificate by the Management Unit/Edge Unit to communicate information to the IoT platform.

The following differentiation is expected depending on the protocol used:

Protocol	Authentication
MQTTS	Username + password
SFTP	Username + password
HTTPS	JWT token

In addition, an Access Control List will be used to manage authorization for data transmission for a specific sensor/UAD/Management Unit/Edge Unit/etc.

2.1.1 Authentication with JWT Token

In order for the Management Unit/Edge Unit and the IoT platform to exchange data, in the case of HTTPS communication, the JWT authentication token must be included in the HTTP header.

The process for creating the JWT token is as follows:

- Token Request: Before making an API call, the device requests a JWT token from the Identity Provider (IdP)
- If authentication is successful, the authentication server returns a JWT token to the caller
- Using the Token for REST API Calls: Once obtained, the JWT token is included in the HTTP Authorization header of each request. Specifically, the token is passed in the HTTP Authorization header of each request: e.g., GET https://{URL}/api/devices/status Authorization: Bearer eyJhbGciOiJIUzI1NiIs....

2.2 Summary of the Specification

The following table provides a summary, based on the type of data flow, of the protocols and data formats used in the messages exchanged between the Management Unit/Edge Unit and the IoT Platform.

During the implementation phase, specific implementation scenarios will be evaluated.

The table below defines the Quality of Service levels for the MQTT protocol, but this parameter must be configurable and therefore modifiable during the implementation phase without any impact on implementation

Flow	Use Cases	Protocol	Data Format
Alert Management - Alarm Management	Near-Real-Time Alarm Transmission	MQTTS QoS=1	json
Alert Management - Fault Management	Near-Real-Time Fault Transmission	MQTTS QoS=1	json
Data Flow Management - Measurements and Results Algorithms	Submission measurement/ result frequency reduced and payload small size (KB range)	MQTTS QoS=0	json
	Bulk transmission of measurements/results with payloads of medium and large sizes	SFTP; SFTP+HTTPS	CSV+ZIP
Data Flow Management - Snapshot	Sending Images of small Base64 images at reduced frequency	MQTTS QoS=0	json
	Sendingbulk images of medium and large-sized	SFTP; SFTP+HTTPS	ZIP images
Device Management	Send commands	MQTT QoS=1	json

The document outlines the definition and methods of data exchange; however, the definition of field domains (i.e., master data) and the associated data validation logic are not within the scope of these specifications and will be defined during the implementation phase between RFI and the field systems supplier. Furthermore, this master data must subsequently be communicated to FS Technology.

The frequency of data transmission to the IoT Platform must be defined on a case-by-case basis between RFI and the supplier. *FS Technology must be involved to assess the impact on the IoT Platform.*

2.3 Protocols

2.3.1 MQTTS Protocol

The MQTTS Protocol supports real-time and/or near-real-time communication between the Management Unit/Edge Unit and the IoT platform in both directions, as a lightweight messaging protocol designed for scenarios requiring low data overhead and/or limited bandwidth. The adoption of this protocol therefore aims to standardize communication with the various field systems of suppliers, in addition to ensuring data stability and persistence.

In particular, the MQTTS protocol is an ISO standard capable of ensuring:

- A uniform standard protocol;
- Data stability and persistence;
- Continuous, unsolicited mode (Push);
- Bidirectional asynchronous transmission (from the field to the platform and vice versa);
- Low impact on the network and computing

resources. Details of the selected protocol are provided

below:

- MQTT 5.0 (release date 2019) defines a TCP/IP data transmission protocol based on a publish-subscribe model that operates through a dedicated message broker located on the IoT Platform. If a later version is used, it must be backward compatible with version 5.0
- TLS (Transport Layer Security) is used to encrypt communication between the client and the MQTT broker.
- The port used by the MQTT protocol must be configurable and will be specified during the implementation phase.
- The Quality of Service can be set to 1 or 0 depending on the data stream, and must be a configurable parameter; therefore, during the implementation phase, it must be modifiable without any impact on the implementation.

2.3.2 SFTP Protocol

The SFTP protocol is intended for use in the case of batch data transmission from the Management Unit/Edge Unit to the IoT Platform

The SFTP (Secure File Transfer Protocol) is a network protocol designed for the secure transfer of files over an encrypted connection. Based on the SSH (Secure Shell) protocol, SFTP provides robust authentication and data protection during transmission, preventing interception and tampering.

SFTP is widely used to ensure the integrity and confidentiality of sensitive information, such as operational data, by reducing the risk of unauthorized access and man-in-the-middle attacks.

2.3.3 HTTPS Protocol

This protocol is intended for use when the IoT Platform sends an API request to resend data in the event of a fault or to send control commands to the field component (Device Management). The HTTPS (Hypertext Transfer Protocol Secure) protocol is a communication protocol used for the secure transmission of data over the Internet. It is based on the HTTP (Hypertext Transfer Protocol) protocol but adds a layer of encryption and authentication to ensure the

privacy and security of data exchanged between the client and the server. HTTPS is widely used to protect the confidentiality of sensitive information.

3. Alert Management and Data Flow

The information transmitted from the monitoring system to the platform has been classified in the following **data catalog** (alarms, measurements, faults):

1) **Alarms:** These indicate an abnormal condition for a specific parameter and consist of a True/False status, supplemented by indications of the alarm's increasing severity. These are further classified as:

- Pre-**alert (Events)**: Refers to the pre-alert level with increasing intensity.
- **Alert (Alarm)**: Relates to an abnormal condition of a specific parameter, defined by a True/False logical state and accompanied by indications of the alarm level, with increasing intensity.

The distinction between messages containing event data and those related to alarms is managed through the "AlarmSeverityLevel" field in the JSON, described below. The classification will be determined by RFI during the master data definition phase.

2) **Failures** (or malfunctions): Information regarding the identification and reporting of abnormal operation and/or failure of one or more components of the monitoring system (sensor, data acquisition unit, Management Unit, Edge Unit, switch, router, etc.). This information is interrelated and also supports device management and Device Management in general.
3) **Measurements**: Quantitative data from the measurements acquired by the sensors. This data stream can also be used to transmit algorithm results.
4) **Snapshot**: Images captured of the monitored structure.

3.1 Alarm Management

Regarding "Alarm Management," the messages to be managed from field systems to the IoT platform are:

- Alarm Trigger
- Alarm reset

While those from the IoT platform to the Management Unit are:

- Alarm acknowledged
- Alarm closure

Below is a description of the JSON fields for the aforementioned messages. During implementation, FS reserves the right to increase the number of JSON fields by up to 50% and to update the description and message type listed below without the Contractor being entitled to claim additional compensation.

Field	Description	Message Type
MsgId	Unique identifier of the sent message that allows differentiation of all the received messages	Alarm occurrence Alarm acknowledged Alarm reset Alarm closure

IDGateway	Unique identifier of the Management Unit or Edge Unit sending the information, allowing for quick tracing back to the device in question	Alarm triggered Alarm acknowledged Alarm reset Alarm cleared
SenderId	Unique of of the provider	Alarm triggered Alarm acknowledged Alarm cleared Alarm cleared
Timestamp	Alarm timestamp of the alarm (timestamp when the problem actually occurred the problem occurred). Format: Milliseconds since Unix Epoch	Alarm onset Alarm acknowledged Alarm reset Alarm closed
Version	Software version	Alarm triggered Alarm acknowledged Alarm reset Alarm cleared
SensorAlarm{AlarmId, Value}	Where AlarmId: of the alarm associated with the asset Value: Value measured by the measuring device: “1” for opening	Alarm triggered
SensorAlarmClosed{AlarmId, Value}	Where AlarmId: Unique identifier of the alarm for the given asset Value: Value assumed by the device measuring : generally “0” for reset	Alarm reset
SensorAlarmHandled{AlarmId, Value}	Where AlarmId: Unique identifier for the alarm associated with the asset Value: Value assumed by measuring device, “2” for acknowledgment	Alarm acknowledged
SensorAlarmManagedClosed{AlarmId, Value}	Where AlarmId: Unique identifier of the alarm for the given asset Value: Value read by the measuring device: “3” for manual	Alarm closure
CurrentValue	Current value (above threshold) of the individual alarm. Provides information on how much the threshold has been exceeded and to perform further processing	Alarm occurrence Alarm acknowledged Alarm reset Alarm closure
Area	Field identifying the alarm type (LV, TLC, TE, IS)	Alarm triggered Alarm acknowledged Alarm reset Alarm closure
FaultTimestamp	Timestamp of the alarm (timestamp of message sent to IOT). Format: Milliseconds since Unix Epoch	Alarm triggered
ProblemIntensity	Text field where the description of the intensity is entered. Field to map in the master data	Alarm occurrence Alarm acknowledged Alarm reset Alarm closure

ProblemLocation	Text field describing the location of the received alarm, provides descriptive and additional information to the Latitude and Longitude fields. Field to be mapped in the master data	Alarm occurrence Alarm acknowledged Alarm return Alarm closure
AlarmText	Free-text field where you can add additional information if necessary	Alarm Trigger Alarm acknowledged Alarm reset Alarm cleared
AlarmSeverityLevel	Identifier of the severity level of the generated alarm	Alarm triggered Alarm acknowledged Alarm cleared Alarm closure
SensorList [{UAD ID, IdSensor, SensorType }]	List of sensors that triggered the alarm (minimum 1) UAD ID: Identifier of the UAD to which the sensor belongs IdSensor: Unique identifier of the sensor that triggered the alarm SensorType: Identifier of the sensor type	Alarm activation Alarm acknowledged Alarm reset Alarm cleared
SpanId	SAP identifier of the span of the structure affected by the alarm.	Alarm triggered Alarm acknowledged Alarm cleared Alarm closure
Pkalarm	Unique identifier of an alarm. Generated by the field system to distinguish identical repeated alarms (same pkalarm) from alarms in different sequences (different pkalarm)	Alarm occurrence Alarm acknowledged Alarm reset Alarm cleared

Table 1 Application/JSON for alarm management

Alarms with the same primary key (Pkalarm) must not be resent to the IoT Platform, unless a resend operation is explicitly requested. However, they must be transmitted if they undergo a status change, such as transitioning from "Alarm Occurrence" to "Alarm Closure," or a change in severity.

Furthermore, in the event of an alarm involving multiple spans, the field systems must send an alarm message for each affected span. Each message will have the same pkAlarm and will contain the sensors affected by the alarm (via the SensorList array) that correspond to the specific span (spanId).

3.1.1 Periodic Alarm Resynchronization

A data flow from the field to the IoT platform must also be provided for the periodic resynchronization of alarm statuses, configurable with the same characteristics as the data flows described above. The details will be defined during the implementation phase.

The process is based on the use of the MQTT protocol for transmitting and receiving messages between the Management Unit/Edge and the central IoT platform.

Process Phases:

1. Submitting a realignment request: The IoT platform publishes an MQTT message with a JSON payload containing the time interval for which it requests the alarm status. The JSON payload has the following structure:
    a. IDGateway: Identifier of the Management Unit/Edge Unit
    b. startTimestamp: Start timestamp of the period to be analyzed.
    c. endTimestamp: End timestamp of the requested period
    d. ValueStatus: Alarm status. Value (0, 1, 2, 3 as described). Blank if all.
2. Request processing: The Management Unit/Edge Unit receives the request and retrieves the alarm information stored for the specified period.
3. Sending the response with the alarm status: The Management Unit/Edge Unit publishes MQTT messages containing the updated status of the alarms detected within the specified time interval and in the specific topic. The message payload is in JSON format and includes the fields described in the previous section (*3.1 Alarm Management*)

3.2. Fault Management

Regarding **"Fault Management**," the messages to be managed from the Edge to the IoT platform are:

- Fault Occurrence
- Fault Resolution

While those from the IoT platform to the Management Unit are:

- Fault acknowledged
- Fault closure

Below is a description of the fields and the required values for each message type. During implementation, FS reserves the right to increase the number of JSON fields by up to 50% without the Contractor being entitled to claim additional compensation.

Field	Description	Message Type
MsgId	Unique identifier of the sent message that allows us to distinguish all received messages	Fault occurrence Fault resolution Fault acknowledged Fault closed
IDGateway	Unique identifier of the Management Unit or Edge Unit sending the information, allowing for quick tracing back to the affected device	Fault occurrence Fault resolution Fault acknowledged Fault closed
SenderId	Unique of of supplier (to be recorded/mapped in the registry)	Fault occurrence Fault resolution Fault acknowledged Fault closed
Timestamp	Fault submission timestamp (Milliseconds since Unix Epoch)	Fault occurrence Fault resolution Fault acknowledged

		Fault closure
FaultTimestamp	Fault timestamp (milliseconds) (since the Unix Epoch)	Failure onset
Area	Field identifying the type of fault (LV, TLC, TE, IS)	Fault onset Fault resolution
FailureText	Free text field where additional information can be added if necessary	Fault occurrence Failure resolution Failure acknowledged Failure closed
FailureSeverityLevel	Identifier of the severity level of the generated failure (to be recorded/mapped in the master data)	Failure onset Failure resolution Failure acknowledged Failure closed
Version	Software version	Fault occurrence Fault return Fault acknowledged Fault closed
SensorList [{UAD ID, Sensor ID, Sensor Type}]	List of sensors that generated the fault, accompanied by lat and lon (minimum 1) IdUAD: Reference UAD IdSensor: Unique identifier of the sensor that generated the fault SensorType: Identifier of the sensor's type	Fault occurrence Fault Return Fault Acknowledgment Fault Closure
SensorFailure { Failure Id, Value}	Identifies the notification of a failure sent to the IoT Platform FailureId: Unique identifier of the failure for the given asset. Represents the ID generated by the control unit and assigned to the specific failure; the same ID will be used throughout the entire failure management cycle (to be recorded/mapped in the master database) Value: 1=indicates the opening	Occurrence of failure
SensorFailureHandled {FailureId,Value}	Identifies the notification of a failure being logged on the IoT Platform FailureId: unique identifier of the failure for the given asset. Represents the ID generated by the control unit and assigned to the specific failure; the same ID will be used throughout the entire failure management cycle (to be recorded/mapped in the master database) Value: type of resolution (value 2 indicates acknowledgment)	Failure acknowledged

SensorFailureClosed {FailureId,Value}	Identifies the resolution of a failure from the Field System to the IoT Platform FailureId: unique identifier for the failure associated with the asset. This represents the ID generated by the control unit and assigned to the specific failure; the same ID will be used throughout the entire failure management cycle (to be recorded/mapped in the master database) Value: type of resolution (value 1 indicates resolution)	Failure resolution
SensorFailureManagedClosed {FailureId,Value}	Identifies the closure of the failure in the management system FailureId: unique identifier of the failure for the given asset. Represents the ID generated by the control unit and assigned to the specific failure; the same ID will be used throughout the entire failure management cycle (to be recorded/mapped in the master database) Value: type of closure (value 3 indicates operator closure)	Fault closure
SpanId	SAP ID of the span of the structure affected by the alarm.	Alarm triggered Alarm acknowledged Alarm reset Alarm cleared
Pkfailure	Unique identifier for a failure. Generated by the field system to distinguish between identical repeated failures (same pkfailure) and failures in different sequences (different pkfailure)	Fault occurrence Fault return Fault acknowledgment Fault closure

Table 2 Application/JSON for fault management

Faults with the same primary key (Pkfailure) must not be resubmitted to the IoT Platform, unless a realignment operation is explicitly requested. However, they must be transmitted

in the event of a status change, such as the transition from "Failure Occurrence" to "Failure Closure," or a change in severity.

Furthermore, in the event of a failure affecting multiple spans, the field systems must send a failure message for each affected span. Each message will have the same pkFailure and will contain the sensors affected by the failure (via the SensorList array) that correspond to the specific span (spanId).

3.2.1 Periodic Fault Resynchronization

A flow (from the field to the IoT Platform) for periodic realignment of the failure status must also be provided, configurable with the same characteristics as the flows described above. Details will be defined during the implementation phase.

The process is based on the use of the MQTT protocol for transmitting and receiving messages between the Management Unit/Edge and the central IoT platform.

Process Phases:

1. Submitting a realignment request: The IoT platform publishes an MQTT message with a JSON payload containing the time interval for which it requests the fault status. The JSON payload has the following structure:
    a. IDGateway: Identifier of the Management Unit/Edge Unit
    b. startTimestamp: Start timestamp of the period to be analyzed.
    c. endTimestamp: End timestamp of the requested period
    d. ValueStatus: Fault status. Value (0, 1, 2, 3 as described). Blank if all.

2. Request processing: The Management Unit/Edge Unit receives the request and retrieves the fault information stored for the specified period.
3. Sending the response with the fault status: The Management Unit/Edge Unit publishes MQTT messages containing the updated status of faults detected within the specified time interval and in the specific topic. The message payload is in JSON format and includes the fields described in the previous section (*3.2 Fault Management*)

3.3 Measurement Management

For this component, two data transmission modes must be provided: PUSH mode and PULL mode.

The primary data transmission mechanism in PUSH SFTP mode will be secure for all types of measurements; optionally, the MQTT protocol may be used for measurements with small payloads (in the order of KBs). For larger payloads, the use of the MQTT protocol is not recommended, as is the case for current dynamic measurements with sizes of several GBs.

The secondary mechanism (which is triggered when the system goes down to retrieve measurements that were not sent during the outage) is a PULL-type mechanism and consists of the Management Unit/Edge Unit exposing a RESTful HTTPS service that can be queried by the IoT platform as needed to request data collected within a specific time window.

This service will allow the Management Unit/Edge Unit to process the request, and the subsequent transmission of the files will be carried out via the SFTP protocol. A description of the process is provided in Chapter *3.3.1, PULL Mode*.

The following is a description of the fields contained in the JSON data transmitted by the Management Unit/Edge to the IoT Platform. During implementation, FS reserves the right to increase the number of JSON fields by up to 50% and to update the description and message type listed below without the Contractor being entitled to claim additional compensation.

Field	Description
MsgId	Unique identifier of the sent message that allows us to distinguish all received messages
IDGateway	Unique identifier of the Edge Gateway (Management Unit/Edge Unit) that sends the information, subject
SenderId	Unique identifier of the provider (to be registered/mapped in the master data)
SpanId	SAP identifier of the bridge span
SensorTimestamp	Timestamp of the recorded measurement. Format: Milliseconds since Unix Epoch
SenderFsamp	Sampling frequency of the data
Timestamp	Timestamp of the message send measurement
Version	Software version
Period	Message transmission period (equals 1 for on-change transmission)
Measures[{UAD ID, Sensor ID, Value, Type, UMSensorValue, axis}]	Array of measurement values recorded by the measuring device IdUAD: Unique identifier of the UAD to which the sensor is attached IdSensor: Unique identifier of the sensor UMSensorValue: Unit of measurement for the sent value Value: Measurement value Axis: Code of the measurement axis Type: Description of the physical quantity
MeasureTypeId	ID to identify the final recipients of the measurement, not mandatory (to be recorded/mapped in the master data)

Table 3 Application/JSON for measurement management

For data transmission via SFTP, a CSV file (within a ZIP file) must be generated with the same fields, where each element of the Measure array will generate a row in the CSV.

The structure of the CSV file may, however, be tailored to the measurements sent and will be defined in detail via RFI during the implementation phase.

The files you submit must follow this naming convention:

- *<assetCode>_<ManagementUnit/EdgeUnit>_<yyyy-mm-dd_hh-mm-ss>_<measurementType>_ <transferId>.csv*
- *<assetCode>_<ManagementUnit/EdgeUnit>_<yyyy-mm-dd_hh-mm-ss>_<measurementType>_ <transferId>.zip*

Below is a description of the individual placeholders:

- *assetCode*: SAP identification code of the monitored infrastructure
- *ManagementUnit/EdgeUnit*: unique identifier
- *yyyy-mm-dd_hh-mm-ss*: timestamp of the file submission
- *measurement_type*: type of measurement contained in the file
- *transferId*: unique identifier of the transmission corresponding to the value returned in the output by the */dfm/device/{IDGateway}* service

The data must be organized into files and directories by type, in accordance with the format and procedures that will be reviewed with RFI during the implementation phase.

This file creation will be governed by a file-locking mechanism, which will be discussed with the FS group during the implementation phase.

In addition, data transfer will be verified using checksums.

If the Management Units/Edge Units of a particular provider contain data processing algorithms that RFI deems useful to incorporate into the IoT platform, RFI reserves the right to integrate the transmission of the results using the same data flow by appropriately modifying the payload and transmission method during the implementation phase.

3.3.1 PULL Mode

Each Edge Unit/Management Unit must have a RESTful service with an API for retrieving measurements, in the event of malfunctions detected in the primary mode (PUSH) or specific requests from the IoT platform, and which takes as input the type of measurement and the sampling time window.

Since the volume of output data cannot be estimated in advance—as the time window could be arbitrarily long—provision has been made to send the requested measurements via the SFTP protocol.

This chapter describes the operating logic and interfaces of the services used to transfer measurement data from the Management Unit and Edge Units to the central IoT platform, based on the device and a time interval specified at the input.

The process is as follows:

1. **Start of transfer**
   - The central IoT platform initiates the process by calling a GET service (/dfm/device/{IDGateway}) exposed by the Management Unit or the Edge Unit.
   - In response, the Management Unit provides:
     - A unique case ID (transferId).
     - Case status
2. **CSV file generation**
   - The Management Unit or Edge Unit, in asynchronous mode, initiates an internal process to retrieve data based on the requested search criteria and with reference to the transferId
3. **File transfer via SFTP**
   - After the file is generated, the Management Unit/Edge Unit establishes an SFTP connection to the central IoT platform to transfer the CSV file (contained in a .zip file) to a specified path
   - The generated CSV file must contain the same fields described in Chapter *3.3 Measurement Management*.
4. **File Integrity Verification via Checksum**
   - To ensure greater control during the transfer, you must calculate the file's checksum before sending it. Once the transfer is complete, you must recalculate the checksum on the received file and compare it with the initial one. If the values match, the transfer was successful and the file was not altered; in this case, proceed to the next step
5. **Completion Notification**
   - Whenever the Management Unit/Edge Unit sends a measurement file, a "pilot file" must first be created in the same destination directory,

named the same as the data file it refers to, except for the extension, which must be ".lock"

- This file must be empty and will serve as a placeholder to indicate that the corresponding measurement file is still being written and cannot be read. Once the write operation is complete, the placeholder file (.lock) must be deleted to allow the full file (.zip) to be read

The REST service interface is described below: dfm/device/{IDGateway}.

The REST service exposed by the Edge Unit/Management Unit has the following interface:

- **http-method**: GET

- **url-path**: /dfm/device/{IDGateway}

- IDGateway: represents the identifier of the Management Unit/Edge Unit

The input values for the search time window must be entered within the **query-parameters** structure**.** Here is a description of each:

Field	Description
startTimestamp	Timestamp of the start date of the data retrieval time window
endTimestamp	Timestamp of the end date of the data recovery time window
type	Type of requested measurement

The output payload is in JSON format and contains the following fields:

Field	Description
transferId	Transfer ID
status	A value indicating whether the file transfer was accepted or not
path	Represents the path where the ZIP file will be saved on the central IoT platform
fileName	File name including the extension
errorMessage	Error description. Set if the call

Below is an example of a correct response (http-status=200) from the Management Unit/Edge Unit:

{

"transferId": "12345", "status":
"accepted",
"path": ""{root}/dfmfilename.csv
"fileName": "filename.csv", "errorMessage
": ""
}

Files must follow the naming conventions described in section 3.3

3.4 Snapshot Management

For this component, two data transmission modes must be provided: PUSH mode and PULL mode, similar to those used for measurements.

The primary transmission mechanism for PUSH SFTP mode will be secure, using the MQTT protocol for images of small size, provided they are Base64-encoded; if, however, the images are large, transmission must occur via SFTP protocol (with zip compression and image resolution to be defined).

The secondary mechanism (which is triggered when the system goes down to retrieve images that were not sent during the outage) is a PULL-type mechanism. It involves the Management Unit/Edge Unit exposing a RESTful HTTPS service that the IoT platform can query as needed to request the images collected within a specific time window.

As for "**Snapshots**," the JSON messages sent to the IoT Platform transmit the image, encoded in Base64. Transmission will be periodic, at set intervals (once or more times a day). Additionally, the payload structure is designed to receive multiple snapshots in a single message.

Below is a description of the fields in the JSON message for snapshot management:

Field	Description
MsgId	Unique identifier of the sent message that allows differentiation of all received messages
IDGateway	Unique identifier of the Management Unit/Edge Unit, allowing for the relevant device
SpanId	Unique SAP identifier of the span
IdUAD	Unique identifier of the UAD
SenderId	Unique supplier identifier
Timestamp	Timestamp of message transmission Format: Milliseconds since Unix Epoch
Version	Software version
Period	Message forwarding period
Snapshots	An element that can contain multiple images (minimum 1)
SnapshotId	Unique identifier of the received snapshot
SnapshotValue	Base64 value of the received image
SnapshotTimestamp	Timestamp of the received image
CameraId	ID of the camera associated with the Sender

3.4.1 Bulk image transmission

For transferring images from the Management Unit/Edge unit to the central platform, in the case of medium/large-sized images, two distinct modes are available:

1. **Batch Mode**
   - Images are sent in **batch** mode according to a predefined schedule.
   - Files are sent via **SFTP**, ensuring secure file transfer from the Management Unit/Edge to the IoT platform. Notification of file transfer completion is described in the "Completion Notification" section of the following transfer method.
   - File Integrity Verification via Checksum**:** To ensure greater control during the transfer, the file's checksum must be calculated before sending. Once the transfer is complete, the checksum of the received file must be recalculated and compared with the initial one. If the values match, the transfer was successful and unaltered, and in this case, the process proceeds to the next step.
   - Completion Notification:

- The Management Unit/Edge Unit must first create a "pilot file" in the same destination directory, named the same as the data file being transferred but with the ".lock" extension
- This file must be empty and will serve as an indicator that the corresponding measurement file is still being written and cannot be read. Once the write operation is complete, the lock file (.lock) must be deleted to allow the full file (.zip) to be read

2. **On-Demand Mode**
   - The central platform initiates the transfer by calling a **GET** service (dfm/device/snapshot/) exposed by the Management Unit/Edge unit. This call triggers the process of retrieving and sending the images**.**
   - Once the process is initiated, the images are transferred asynchronously via **SFTP**. It is important to note that the images must be included in a .zip file before being sent.
   - File Integrity Verification Using a Checksum**:** To ensure greater control during the transfer, you must calculate the file's checksum before sending it. Once the transfer is complete, you must recalculate the checksum on the received file and compare it with the initial one. If the values match, the transfer was successful and the file has not been altered; in this case, proceed to the next step.
   - Completion Notification:
     - The Management Unit/Edge Unit must first create a "pilot file" in the same destination directory, named after the data file being transferred but with the ".lock" extension
     - This file must be empty and will serve as a placeholder to indicate that the corresponding measurement file is still being written and cannot be read. Once the write operation is complete, the placeholder file (.lock) must be removed to allow the full file (.zip) to be read

The REST service interface is described below: dfm/device/snapshot/{CameraId}.

The REST service exposed by the Management Unit/Edge Unit responsible for initiating image acquisition has the following interface:

- **HTTP method**: GET

- **url-path**: dfm/device/snapshot/{CameraId}

- CameraId: represents the unique identifier of the camera

The input values for the search time window must be entered within the **query-parameters** structure**.** Here is a description of each:

Field	Description
startTimestamp	Timestamp of the start date of the data retrieval time window
endTimestamp	Timestamp of the end date of the data retrieval time window

The output payload is in JSON format and contains the following fields:

Field	Description

transferId	Transfer identifier
status	A value indicating whether the file transfer was accepted or not
path	Represents the path where the ZIP file is saved on the central IoT platform
fileName	File name including the extension
errorMessage	Error description. Set if the call

Below is an example of a correct response (http-status=200) from the Management Unit:

{

"transferId": "12345", "status":
"accepted",
"path": ""{root}/dfm/snapshot",",
"fileName": "filename.zip", "errorMessage":
""

}

In both cases (PUSH/PULL), the .zip file transferred from the Management Unit/Edge Unit to the IoT platform must follow the following structure for each image sent:

- A metadata file containing the following fields: *MsgId, IDGateway, SenderId, Timestamp, Version, SnapshotId, SnapshotTimestamp, NameFile, CameraId*. For a description, refer to the information provided in Chapter 3.4.
- File containing the captured image.

The file naming convention is as follows:

- Acquired image file: *<assetCode>_<managementUnit/EdgeUnit>_<yyyy-mm-dd_hh-mm-ss>_ <CameraId>.<format>*
- File metadata metadata: *<assetCode>_<ManagementUnit/EdgeUnit>_<yyyy-mm-dd_hh-mm-ss>_<transferId>_metadata.csv*
- File zip: *<assetCode>_<ManagementUnit/EdgeUnit>_<yyyy-mm-dd_hh-mm-ss>_ <CameraId><transferId>.zip*

3.5 Topics to use

Below are examples of the 5 MQTT topics to which messages are expected to be received. The corresponding legend is provided:

CENTRAL/TR0001-XX-XXXX-XXX-XXX/ALARM/<GatewayID>

CENTRAL/TR0001-XX-XXXX-XXX-XXX/ALARM/DEVICE/<GatewayID>

CENTRAL/TR0001-XX-XXXX-XXX-XXX/MEASURES/<IDGateway>

CENTRAL/TR0001-XX-XXXX-XXX-XXX/FAILURE/<IDGateway>

CENTRAL/TR0001-XX-XXXX-XXX-XXX/FAILURE/DEVICE/<IDGateway>

CENTRAL/TR0001-XX-XXXX-XXX-XXX/SNAPSHOT/<IDGateway>

Where:

- CENTRAL: reference department code. (This may be RM for the DOIT in Rome, BA for the DOIT in Bari, and so on).
- TR0001-XX-XXXX-XXX-XXX: INRETE2000 reference bridge code, fifth-level technical site.
- ALARM/<IDGateway>: identifies the topic to which alarm triggers and clearances are sent from field systems to the IoT platform;
- <IDGateway>: gateway ID, i.e., the Edge Unit/Management Unit selected for illustrative purposes, which is replaceable and subject to agreement. It uniquely identifies the device integrated with the IoT Platform. This topic is used for communication from the Management Unit to the IoT Platform
- ALARM/DEVICE/<IDGateway>: defines the topic to which acknowledgment and alarm clearance messages are sent from the IoT Platform to the field systems.
- MEASURES/<IDGateway>: identifies the topic to which measurements are sent.
- FAILURE/<IDGateway>: This topic is used to send failure notifications.
- SNAPSHOT/<IDGateway>: Specifies the topic to which snapshots are sent.

4. Device Management

The Device Management component offered by the IoT Platform plays a fundamental role in managing field devices within the monitoring system. Thanks to its features, it allows you to configure and remotely control devices (sensors, UADs, Management Units, and Edge Units) according to operational needs, monitor their status using advanced diagnostic tools, and manage them efficiently.

4.1 Master Data Management

In order for the IoT Platform to provide device management capabilities, it must manage the device registry, through which it is possible to register, update, and manage sensors, DACs (Data Acquisition Units), and Edge Management Units, as well as obtain the coordinates needed to correlate IoT data with the global asset identification codes (e.g., SAP codes for bridges).

Global identification codes are always defined centrally and must be communicated to suppliers to correctly map local IoT devices.
With this feature, users can maintain an up-to-date inventory of devices, track their development over time, and ensure efficient management of the entire IoT ecosystem.

For a device to send data to the IoT Platform, it must first be registered in the registry (Access Control List). If a Management Unit, an Edge Unit, or another entity interfacing with the IoT Platform is not registered, the platform will not accept the transmitted data, thereby ensuring strict control over the integrity and security of the information managed by the system.

Device registration in the Device Registry module will be managed via an offline process, following the acceptance testing of the sensor installation; the content and procedures will be agreed upon with RFI.

Each Management Unit and Edge Unit must have network endpoints associated with them for the HTTPS protocol (HTTPS Endpoint) required to route traffic to them globally. These endpoints must be recorded in the master database.

The geographic location of all elements comprising the field network must also be recorded in the master database. In any case, all attributes to be recorded in the master database will be defined in detail during the implementation phase.

Upon powering on the sensor, the corresponding Management Units will send a LOGIN message with the details defined in the following paragraph.

4.2 Command Management

The main management operations that can be performed include:

- Login: Registration of sensors/UADs/Management Units/Edge Units
- Restart: Restart sensors/UADs/Management Units/Edge Units
- Stop: Stop sensors/UADs/Management Units/Edge Units
- Send configurations regarding data logging periods, trigger and cutoff thresholds, pre-trigger and post-trigger window durations, and sampling frequencies
- Request current monitoring configuration and request a diagnostic report/health check on the operational status of devices (e.g., Sensors, UADs, Management Units, Edge Units)
- Time synchronization, e.g., NTP (Network Time Protocol)
- Firmware update
- Disabling a group of sensors to ignore their alarms and detected data
- Reset the sensor group to resume receiving alarms and sensor data

FS reserves the right to modify and/or increase the scope of the above-mentioned operations by 50% without the Contractor being entitled to claim additional compensation. Should the supplied IoT systems have additional device management features, these must be made available for integration into the Device Management system in order to better organize the remote control processes.

To ensure these features are implemented, the Management Unit and the Edge Unit must be equipped with a software module that implements the MQTT Publisher-Subscriber model, developed by the vendor, capable of:

- correctly interpret commands issued remotely via MQTTS (Command request)
- send an acknowledgment of receipt via an MQTTS ACK message (Command response), specifically populating the JSON field of the response: *Response {DeviceId, Success, Value}*
- execute the requested actions based on the received command.
- notify with a message MQTTS the new status of device, by appropriately evaluating the JSON field of the response: SensorsStatus *{DeviceId, Value}*

The payload of the REQUEST and RESPONSE messages is in JSON format and follows the structures shown below.

REQUEST message

Field	Description
MsgId	Unique identifier of the sent message
IDGateway	Unique identifier of the device. Can be set to the Management Unit ID of the Management Unit or the Edge Unit ID

Timestamp	Alarm send timestamp
Version	Software version
Period	Message forwarding period
RequestId	Unique identifier for the command request

Command [{DeviceId, Value, Params[key, value]}]	Command container array object:
	• DeviceId: Identifier of the sensor or UAD to which the command is directed. It can be set to NONE when sending commands to Management Units or Edge Units
	• Value: Value of the sent command to be associated with the sensor/UAD/Management Unit/Edge Unit
	• Params[key, value]: Key-value array for setting variable parameters depending on the command
	  o Key= parameter type
	  o Value= parameter value. This can also be the URL of the repository from which to download the firmware to be installed
	The domain of this array will be defined during implementation

Below is the description and range of values for the Value field in the REQUEST message:

- LOGIN: registration of sensors/UADs/Edge Units/Management Units
- RESTART: scheduled restart
- STOP: Device shutdown
- SETUP: Change monitoring parameters
- STATUS: Request for current monitoring configuration. This class includes monitoring of the operating status of the sensor/UAD/Management Unit/Edge Unit (e.g., battery level, or power on/off)
- SYNC_NTP: NTP time synchronization
- FIRMWARE: Remote firmware update for the device
- EXCLUDE: Disable sensor group to ignore alarms and detected data
- RESTORE: Restore a sensor group to resume receiving alarms and detected data

During implementation, FS reserves the right to increase the number of JSON fields by up to 50% and to revise the logic for populating those fields without the Contractor being entitled to claim additional compensation.

Below is a description of the RESPONSE message:

Field	Description	Message Type
MsgId	Unique identifier of the sent message	ACK message Status message
IDGateway	Unique identifier of the device. Can be set to the ID of the or the Edge Unit ID	ACK message Status message
Timestamp	Alarm Send Timestamp	ACK message Status message
Version	Software version	ACK message Status Message
Period	Message transmission period	ACK message Status message
RequestId	Unique identifier of the command request (see legend above)	ACK Message Status Message

Response [{DeviceId, Success, Value}]	Array of response container objects • DeviceId: Identifier of the sensor or UAD to which the command is directed. It can be set to NONE when sending commands to Management Units or Edge Units • Success: Boolean value (true/false) indicating whether the command succeeded or failed. • Value: Value assigned to the sensor following the execution of the command or the command's identifier	ACK message
SensorsStatus [{DeviceId, Value}]	Array of objects containing the device status following a status change: • DeviceId: Sensor or UAD identifier • Value: value detected by the sensor/UAD The object is optional and is populated when the device sends its new, updated status	Status Message

During implementation, FS reserves the right to increase the number of JSON fields by up to 50% and to revise the logic for populating these fields without the Contractor being entitled to claim additional compensation.

The command sequence must also be configurable

The statuses described in this chapter are all sent in response to request and response commands, with the exception of the RESTART/STOP status, which must also be sent at a configurable interval of x minutes (default 10 minutes).

4.3 Topic to use

CENTRAL/TR0001-XX-XXXX-XXX-XXX /DM/CMD/<IDGateway>

CENTRAL/TR0001-XX-XXXX-XXX-XXX /DM/CMD/FEEDBACK/<IDGateway>

CENTRAL/TR0001-XX-XXXX-XXX-XXX /DM/STATUS/<IDGateway>

Legend

- CENTRAL: reference compartment code. (This may be RM for the DOIT in Rome, BA for the DOIT in Bari, and so on).
- TR0001-XX-XXXX-XXX-XXX SAP code of the Bridge
- The DM/CMD topic for sending Device Management commands from the IoT Platform to the field specifies the topic to which commands are sent.
- DM/CMD/FEEDBACK topic for the response from the field to the IoT Platform to acknowledge the command.
- DM/CMD/STATUS topic for sending the device status

5. Remote access to field devices by RFI

RFI users are expected to be able to access supplier data from the field; each supplier must provide access to its software platforms, located on field units (UADs, management PCs, edge PCs, etc.), during the implementation and operation phases of its field solution. Specifications and user manuals must be provided to enable RFI users to operate the systems.

To ensure the security of corporate networks, firewalls will be implemented to segregate IT networks from OT networks, thereby protecting corporate systems from field operational infrastructure.

6. Remote access to field equipment by Suppliers

Third-party suppliers will be able to access field systems via an IPSec VPN connection specifically configured for access to the RFI network, the configuration parameters of which will be shared during the contract execution phase.

The protocols permitted within the VPN must be encrypted only (e.g., HTTPS, SSH, SFTP, etc.). Unencrypted protocols will not be authorized. The contractor's solution must in any case be described in the design documents provided by the contractor and submitted to RFI for approval.

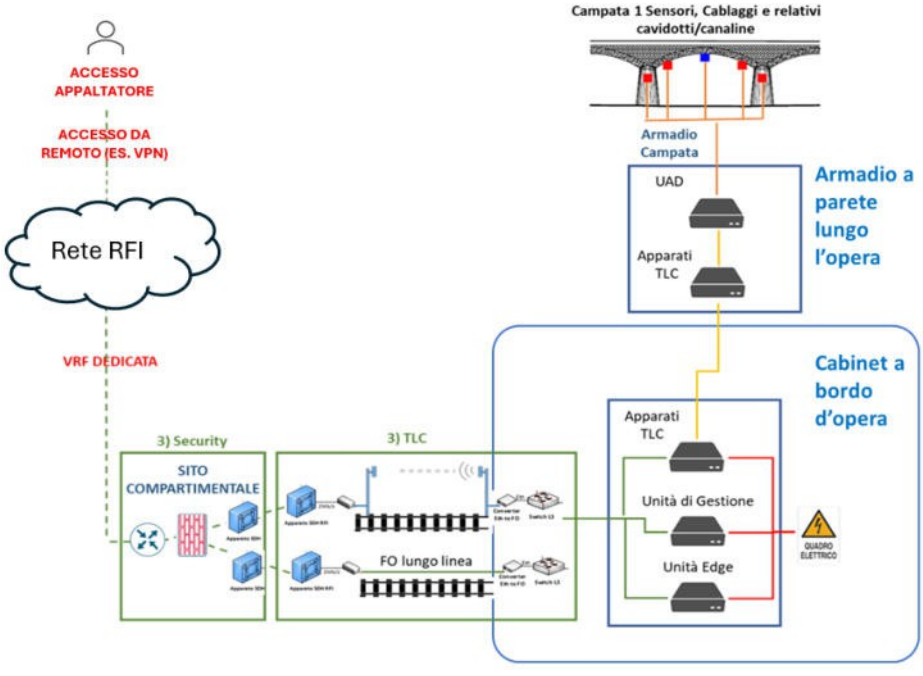

Figure 3 – Remote access to field equipment by Suppliers via fiber optics and the RFI fixed network

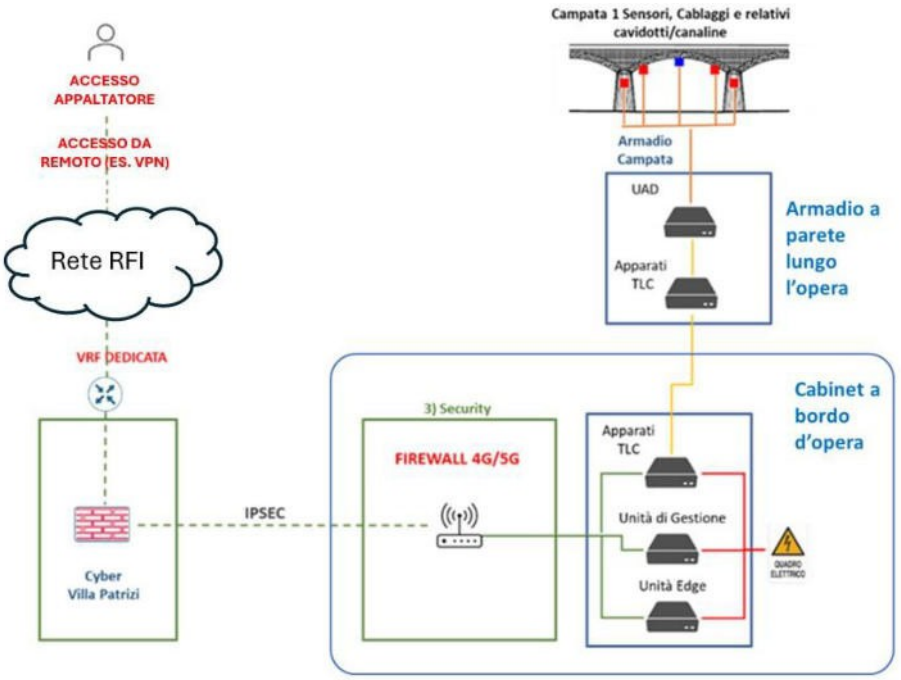

Figure 4 – Remote access to field equipment by suppliers via IPSec VPN over the Internet